

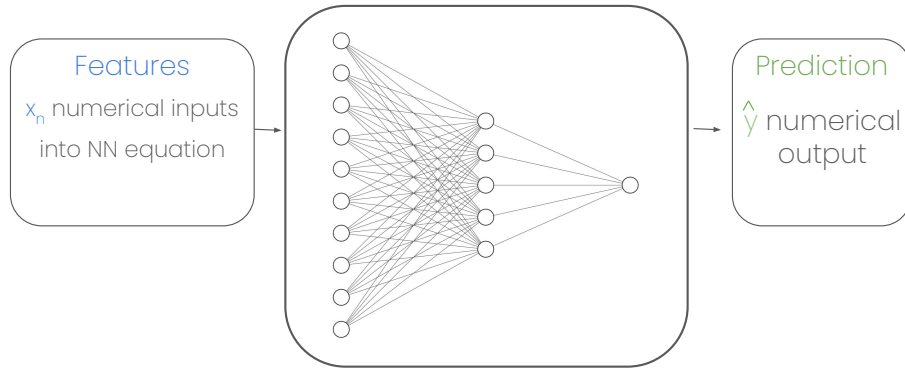
Reinforcement Learning

ZeroToML

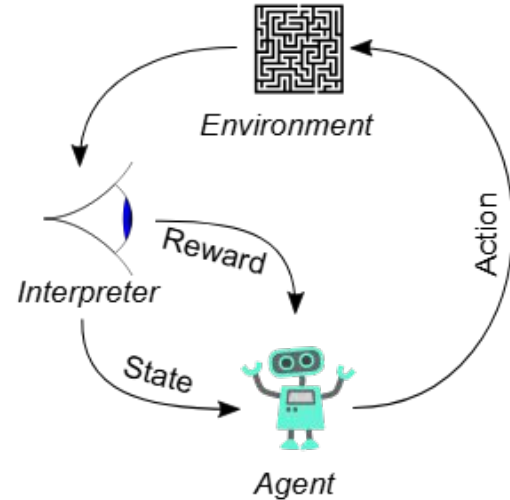


What is Reinforcement Learning?

Supervised Learning



Reinforcement Learning

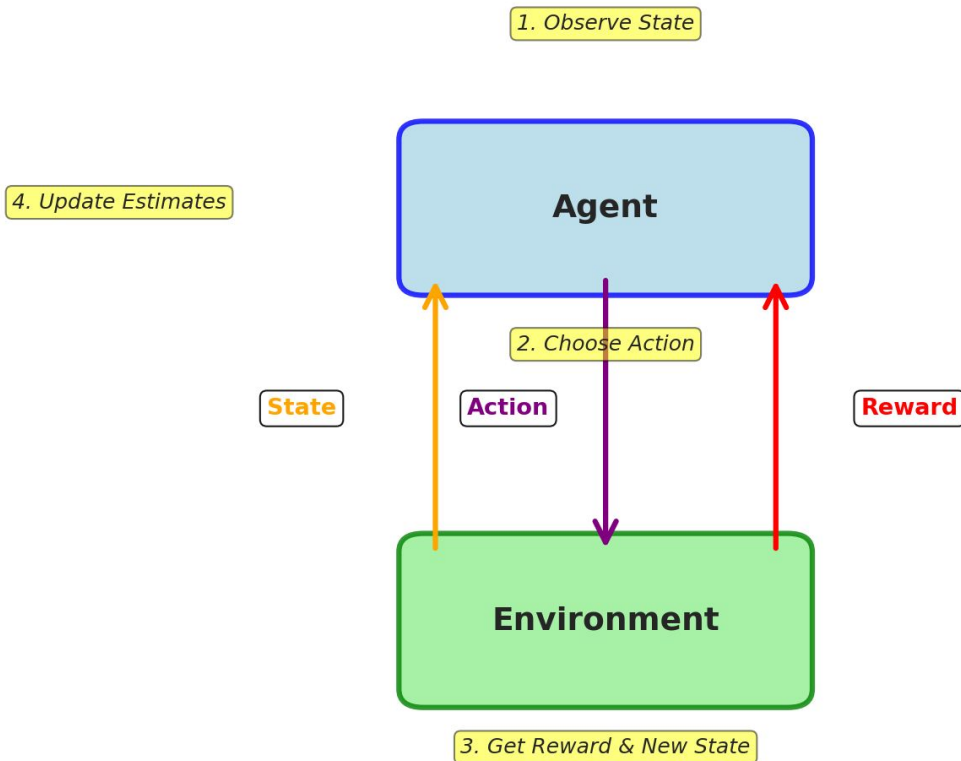


Advantages of Reinforcement Learning

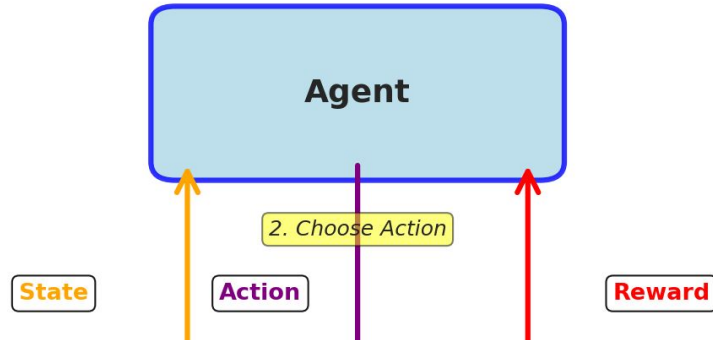
- No data labelled required
- No single right answer



The RL Loop



Deeper look into the agent



Policy: a function that gives the probability of which actions to take



Value Functions:

We don't have **loss functions**, so how do we use instead?

$V(s)$

- State-Value: Value of a **state**

$Q(s, a)$

- Action-value: Value of an **action** given a state

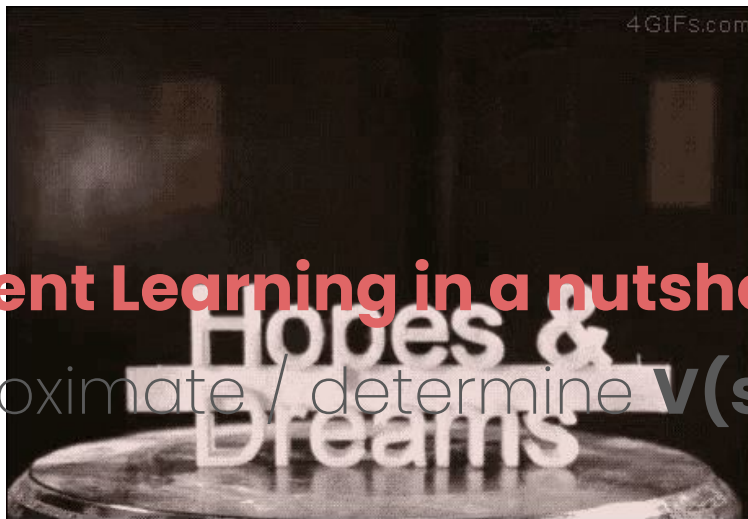


Value Functions:

If we know $V(s)$ and $Q(s, a)$, then we're done 🎉

Reinforcement Learning in a nutshell:

How to approximate / determine $V(s)$ and $Q(s, a)$



Start Simple: A Table

- Each value $Q(s, a)$ is stored in a table cell
- The table is initialized randomly or 0



Start Simple: Multi-Armed Bandit

- What if there's only **one possible state**
 - Ex: Casino with n slot machines, each with a different *unknown* probability of winning

Consider $Q(\mathbf{a})$ instead of $Q(\mathbf{s}, \mathbf{a})$



Optimizing

Update the table based on **rewards**

$$Q_{t+1}(a) = Q_t(a) + \alpha [r_{t+1} - Q_t(a)]$$



Learning Rate



Q-Learning: Adding states

Action value function, $Q(\mathbf{s}, \mathbf{a})$, is a table

		Actions			
		A_1	A_2	...	A_M
States	s_1	$Q(s_1, A_1)$	$Q(s_1, A_2)$		$Q(s_1, A_M)$
	s_2	$Q(s_2, A_1)$	$Q(s_2, A_2)$		$Q(s_2, A_M)$
	$\vdots$			$\ddots$	$\vdots$
	s_N	$Q(s_N, A_1)$	$Q(s_N, A_2)$	...	$Q(s_N, A_M)$



Q-Learning: Introducing time

What if an action has no immediate reward, but **increase future reward**?

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$



Q-Learning: Introducing time

$$Q(s, a) := Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

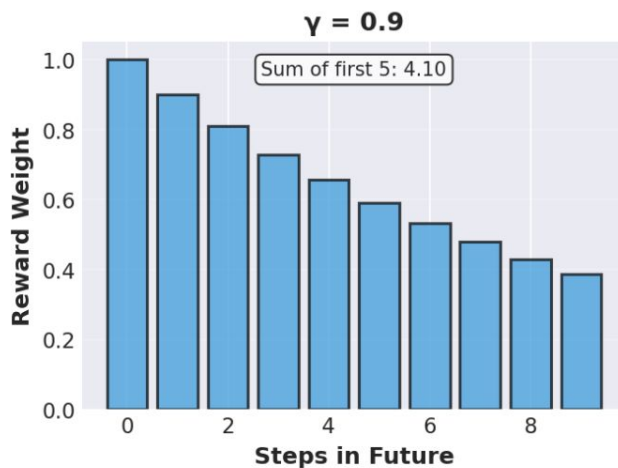
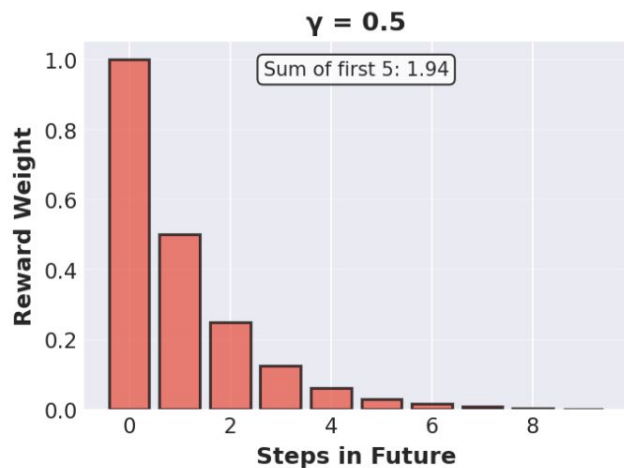


Discount rate



Q-Learning: Discount Factor

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$



Exploration vs. Exploitation

When training, how much do we **explore** vs. **exploit** (follow the Q-table)?

Epsilon-Greedy policy: exploration rate ϵ :

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \arg \max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$$

$1 - \epsilon$ chance to choose the best action



Beyond: Deep Q Networks

Instead of a table, what if we used **neural networks**?

Key idea:

- Q Networks receive state and predicts a Q-value for each action
- Essentially run supervised learning with an artificial label:

$$r + \gamma \max Q(s', a')$$



Notebook



End

